
SHARPIE: A Robust Framework for Streamlining Human-AI Reinforcement Learning Experiments

Libio Goncalvez Braz*
Utrecht University
l.b.goncalvesbraz@uu.nl

Floris den Hengst*
VU Amsterdam
f.den.hengst@vu.nl

Kevin Godin-Dubois*
VU Amsterdam
k.j.m.godin-dubois@vu.nl

Hüseyin Aydın
Utrecht University
Middle East Technical University

Kim Baraka
VU Amsterdam

Frans A. Oliehoek
TU Delft

Shihan Wang
Utrecht University
s.wang2@uu.nl

Abstract

Reinforcement Learning (RL) offers a general approach for modeling and training AI agents, potentially including human-AI interaction scenarios. In this paper, we propose SHARPIE (Shared Human-AI Reinforcement Learning Platform for Interactive Experiments) to address the need for a generic framework to support experiments with RL agents and humans. Its modular design consists of a versatile wrapper for RL environments and algorithm libraries, a participant-facing web interface, logging utilities, deployment on popular cloud and participant recruitment platforms. It empowers researchers to study a wide variety of research questions related to the interaction between humans and RL agents, including those related to interactive reward specification and learning, learning from human feedback, action delegation, preference elicitation, user-modeling, and human-AI teaming. The platform is based on a generic interface for human-RL interactions that aims to standardize the field of study on RL in human contexts. To demonstrate ease-of-use and versatility, we replicate four illustrative experiments from the multi-agent RL literature and report on the platform’s performance under stress testing to showcase its robustness.

1 Introduction

Reinforcement learning (RL) agents learn through environmental interaction to maximise cumulative long-term reward [37]. In recent years, this paradigm has expanded beyond pure autonomous discovery to include scenarios involving humans, under the umbrella term of Reinforcement Learning from Human Feedback [21]. As available compute increases, RL methods are being deployed in increasingly complex domains where humans are central to the loop. RL agents interact with humans in a wide variety of ways: in some problem settings, RL agents observe humans to achieve a common goal or act on their behalf [29]; in other settings, RL agents communicate with humans to provide services or support their decision making [43]; in other settings, human feedback can also be used to evaluate the value of agent behavior [10, 25], to communicate preferences [6], to correct behavior [36], and to guide exploration [18, 40].

*Equal contribution.

These different types of interaction highlight the need to incorporate humans in the training and evaluation of RL agents for various aims including efficiency, safety, personalization and alignment purposes. However, implementing these experiments in a standardized manner presents specific infrastructural challenges. Connecting local RL training loops to human participants requires building interfaces, managing state synchronization, and implementing data logging services. Consequently, current research on RL involving humans frequently relies on custom, ad-hoc point-solutions engineered for individual studies. While popular RL libraries such as Gymnasium [41], PettingZoo [39], JaxMARL [35], and MO-Gymnasium [3] provide established benchmarks in their related domains, experimentation involving humans in RL has thus far seen a lack of standardization and has relied on study-specific platforms and tools. While some efforts have been made, these have been lacking in their integrative scope, and have suffered from support challenges [38].

To address this infrastructural gap, we propose SHARPIE (**Shared Human-AI Reinforcement-Learning Platform for Interactive Experiments**). This platform provides a reusable toolset designed to standardize the design, deployment, and analysis of human-RL interactions. By moving away from rigid point-solutions and adopting a modular platform, researchers can more systematically develop more experimental setups. What is more, through first-class support for multi-agent and multi-human interaction scenarios, we hope to facilitate the study of interactions beyond the basic and unidirectional patterns we largely see today [4] in order to effectively address problems which humans nor agent can solve independently [1, 9].

The proposed platform provides versatile wrappers for popular single-agent RL and multi-agent RL environments and algorithms, supports configurable communication channels between humans and RL agents in various modalities, and offering logging services, deployment utilities, and participant recruitment platforms integration. The platform aims to empower researchers to address research questions on the interaction of RL agents and humans, focusing on interactive reward specification and learning, learning from human feedback, action delegation, preference elicitation, user-modeling, and human-AI teaming. By providing a technical platform in which these paradigms can be easily studied, compared, and combined, we hope to increase rigor and contribute to setting the standard for human-RL experiments.

Our main contributions in this work are outlined below:

- SHARPIE introduces a potential standard for RL-based human-agent interactions in a multi-human/multi-agent setting in the same way that the Gymnasium API has become this standard for fully simulated RL environments.
- SHARPIE supports experiments with diverse RL applications via minimal Gymnasium extensions and native integration of crowdworker platforms.
- four representative case studies are benchmarked to illustrate different modalities human participants can be integrated into the RL learning process.
- Several performance and scalability tests to demonstrate that our framework supports distributed interaction across geographically distant human participants, with end-to-end latency within 0.21 seconds for local area networks.

The rest of the manuscript is organized as follows. Section 2 summarizes the related work for the study. Design principles of SHARPIE framework are explained in section 3. While section 4 provides the qualitative evaluation of SHARPIE with 4 use cases, quantitative analysis for its performance is presented in section 5. Finally, the implications of the study along with possible future research directions are discussed in section 6.

2 Related Work

There are several software platforms for conducting behavioral experiments [12, 30]. None of these offers any infrastructure for any type of artificially intelligent entities, whether learning or not. Various platforms and software packages have been developed for the deployment of RL [2, 15, 45]. However, these are not focused on controlled experiments on the interaction between RL agents and humans.

Several successful platforms for the development of human-in-the-loop RL algorithms and evaluation environments have been made over the past decades, emphasizing a different number of objectives, different numbers of agents, different kinds of tasks and different programming languages. We do

not intend to review all software packages in RL and here we focus only on the related work that particularly deals with RL platforms that explicitly target the study of RL in a context that involves humans.

With recent advances in Reinforcement Learning from Human Feedback (RLHF), platforms such as [10, 42] have been developed. However, the interfaces for human interaction provided by these platforms are only capable of handling restricted feedback, and treat humans mostly as a source of information rather than an interaction partner. Along similar lines, Qian et al. [31] proposed UserRL, an approach for training agentic large language models based on human input with RL in various settings. This work is limited to a *one-to-one* text-based interaction with LLMs whereas we focus on the multi-human multi-agent experiments involving various modalities, including reward annotation, action guidance, as well as textual communication.

With a specific focus on user studies, Knierim et al. [23] presents a framework in human teaching RL agents setting with audio communication. Their codebase also facilitates a multi-participant remote Wizard-of-Oz approach [17], where one participant can play the role of a teacher and another plays the role of a learning agent, addressing additional requirements for early iterative testing of interactive algorithmic settings, where usability and human behavior are used as a baseline, target, or data collection / calibration step. From an algorithmic perspective, although an alternative modality is proposed in this study (namely prosodic voice cues), the interactions are designed to be unidirectional, with human teachers guiding RL-agents with a restricted communication modality.

Closer to the scope of SHARPIE, the HIPPO-GYM platform [38] provides a framework that involves human interactions in a RL setting. While similar in its main objectives as well as the needs it addresses, this framework is limited to single-agent tasks and provides only specific interactions in those tasks (i.e. humans teaching RL agents through various forms of feedback).

On the multi-agent front, previous studies in multi-agent learning such as MAgent [44] and MOMAland [14] are capable of handling multi-agent experiments. MAgent supports a large population of agents, ranging from hundreds to millions, whereas it is possible to train the agents for multiple objectives in MOMAland. However, these packages do not yet support human interaction in any way. SHARPIE specifically targets the gap of easily setting up experiments with (multiple) human participants and RL agents by interfacing with existing environment and algorithm libraries.

Finally, the closest work to SHARPIE that we have identified in the existing literature is Multi-User Gymnasium [28] which is a framework in early stage of development that enables human-AI interaction in multi-user RL settings. This framework currently provides two different setups: single human with multiple agents in a browser based execution and any number of human and AI agents in a client-server based execution of experiments. With its current structure, Multi-User Gym seems to be specific to the Gymnasium environments where human(s) can take the control of RL agents in a game setup which can require more effort to deal with a diverse set of scenarios including various type of human-RL agent interactions. SHARPIE, on the other hand, aims to provide a more generic and extendable design that is able to efficiently facilitate such scenarios.

All in all, there are several existing approaches and subsequently experimental platforms that have looked at specific restricted human-RL scenarios, but do not focus on offering the platform as a versatile tool for research in this area. We envision SHARPIE will empower human-AI researchers to deploy interactive agents and collect data on a wide variety of human-in-the-loop RL scenarios in *rich, real-time and potentially multi-agent* settings.

3 SHARPIE Framework

SHARPIE is designed as a modular, Python-based framework that provides a unified interface wrapping reinforcement learning (RL) environments, algorithms, and human-in-the-loop methodologies. Rather than providing tightly coupled integrations with specific existing software packages, SHARPIE favors a loosely coupled design to maximize compatibility across heterogeneous infrastructure. Furthermore, unlike existing platforms tailored for specialized setups [38], SHARPIE targets a platform-agnostic deployment pipeline. This enables researchers to host experiments locally, on self-managed infrastructure, or via cloud providers without altering the underlying codebase.²

²Code and hosted demo link available at <https://github.com/hybrid-intelligence/SHARPIE>.

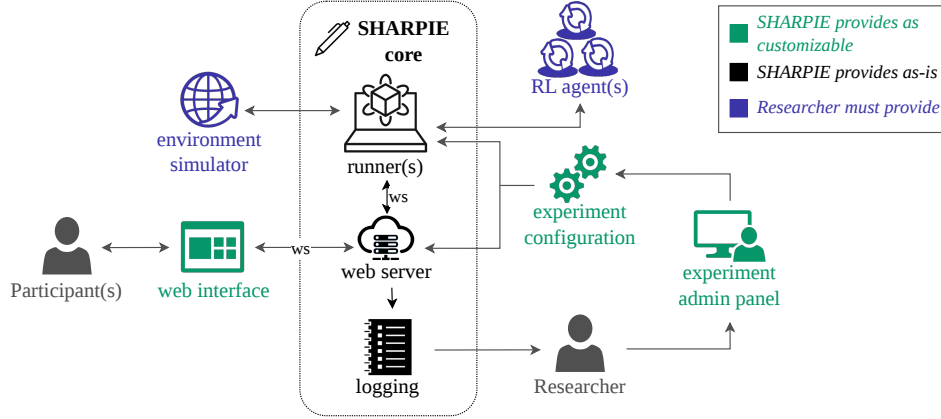


Figure 1: High-level SHARPIE architecture.

SHARPIE operates under three primary assumptions. First, we assume that the interaction loop is modeled as a Markov decision process, or its multi-agent, multi-objective and partially observable extensions. Second, we assume that the adopted environment simulator and agent algorithms implement the de-facto standard Gymnasium API, minimally extended with vectorial rewards and actions for multi-objective and multi-agent scenarios. Third, we assume the presence of at least one human participant or one agent to interact with the environment.

As illustrated in Figure 1, the library is composed of three main families of modules. In black are the built-in components, responsible for scheduling episodes and communication between agent(s), human(s), the learning algorithm(s) and the environment. These core modules implement a client-server-runner architecture that ensures scalability with respect to agents and participants while allowing for high-fidelity synchronization in real-time interaction scenarios with reasonably geographically distributed participants.

A key design decision here is the use of asynchronous participant input handling in combination with the use of web sockets for server-side synchronisation of participant and agent inputs with environment state. All connections indicated ‘ws’ in Figure 1 are implemented using this low-latency bidirectional communication layer over the TCP protocol, and allow for data exchange in various modalities with message-order guarantees, making them suitable to flexibly transmit e.g. environment renderings, text messages, reward annotations etc. In SHARPIE, web sockets implement all interactions between the client web interface, the web server, and the environment and agent runners following a JSON-based API. This brings transparency across interaction channels which aids with observability and debugging.

Messages with a high payload may prove challenging with this type of synchronized communication. Data of substantial size such as large byte arrays can additionally be compressed to reduce the overhead brought about by network communication, in cases of low-bandwidth connections between the web server and participants. Additionally, the synchronization implementation in the web server allows for a reduction of the interaction frequency with the environment. This is done adaptively by the runner, based on the speed at which participant inputs are received to ensure synchronization even in the case of high-latency connections.

The functionality of all core *as-is* modules is supported by a data model suitable for the domain of human-RL experimentation, including experiment configuration, participant registration, experimental data logging, etc., see Figure 7 in Appendix Appendix B. All data is stored in a relational database (DB), following established best practices in the field. For all DB write operations associated with experiment logging, we use in-process buffering and batch writing to optimize the DB payload in high-frequency interaction scenarios, and experienced no bottlenecks in data storage during scalability and performance profiling (see Section 5).

While the components marked in black provide the backbone of the library, they will remain largely transparent to researchers in regular usage of the framework. We now turn to the components that researchers can customize based on their particular research questions of interest. Figure 1 marks

components such components **green**. Here, we first identify the web interface, for which SHARPIE offers a set of user interface (UI) components which are typical for human-RL experiments, such as keyboard button presses, text inputs, reward boxes, and a history of interactions and user inputs. Second, SHARPIE provides functionality to configure and compose experiments via a configuration module, which can be accessed directly using `.yaml` file or using the visual experiment configuration panel accessible to admins via a web interface.

Having looked at the functionality provided as-is and provided for customization, we now list all components that researchers implement themselves within the SHARPIE framework, depicted in **blue**. These correspond to the agents and their environments. SHARPIE here focuses solely on functionality to ensure that any relevant agent or environment implementation can be wrapped by the runner processes, to allow full flexibility in studying the research questions at hand. In practice, all environments and agents that implement a gymnasium API minimally extended with vectorial rewards and actions to support multi-agent environments, can be integrated in the SHARPIE framework. The extension of the gymnasium API consists of vectorial rewards and actions for multi-objective and multi-agent environments and algorithms and can be implemented within a lines of code.

Having looked at all components listed in this figure, we now turn to some design auxiliary design decisions that span the entire SHARPIE framework. Firstly, the SHARPIE framework is designed for platform-agnostic deployment, including deployment to a managed cloud server, a private machine, or a local host. Given the widespread support for Python, this encompasses any desktop operating system (e.g. Linux, Mac, Windows) with Python 3 installed, including remote web-servers or platforms such as Amazon Web Services (AWS). This provides an abstraction between the machines that the researchers and participants are using and where the experiment is actually running. Such a feature is essential in studies involving numerous participants with varying locations and hardware. Delegation is done through the runners, allowing replicates of an experiment to be performed in parallel by instantiating multiple such runners on a machine, or even running multiple geographically distributed SHARPIE instances simultaneously. We report scalability results on a single machine in section 5, after providing the case studies for functional validation in the following section. Utilities and documentation for these types of deployment are provided, including how to set up experiments with a crowdworker platform.

4 Functional Validation via Case Studies

We validate the expressiveness and flexibility of the proposed framework by replicating multiple interactive scenarios involving RL agents and human participants. To evaluate whether the framework is sufficiently expressive to implement a wide range of such scenarios, we consider a taxonomy of interaction paradigms. Figure 2 displays the different interaction paradigms based on how humans, agents and the environment can interact with one another based on related work categorizing paradigms in which humans learn in interactions with humans [5, 21, 33].

In the first interaction paradigm, RL agents and humans can send *messages*. These messages can be used to give instructions, ask for help, explain the agent internal state, give action advice, etc. and can have various modalities including text, images, etc. In the second paradigm, humans can select and override RL agent actions. This paradigm includes shared control, learning from demonstrations etc. In the third paradigm, humans act as agents in the environment. This paradigm covers shared tasks, human-AI-teaming etc. In the fourth paradigm, the human specifies or alters an environment reward, which corresponds to various evaluative feedback and reward shaping setups. These paradigms are not mutually exclusive and can in practice be combined.

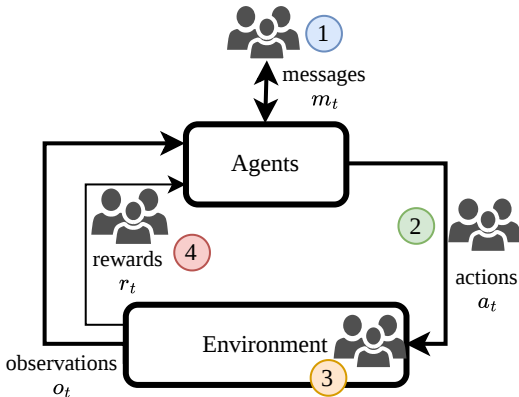


Figure 2: Taxonomy of agent-human-environment interactions.

Table 1: Use cases included in functional validation. The last columns indicate the numbers of lines of code (LoC) and database (DB) entries to integrate with SHARPIE.

Algorithm	Paradigm	Description	LoC	DB
SayCan [20]	1 messages	Language instructions, learned affordances	17	5
Dagger [34]	2 actions	Queries demonstrations for visited states	65	5
SP/BC [9]	3 environment	Plays cooperatively with a human	29	6
TAMER [25]	4 rewards	Learns from human evaluative feedback	31	5

Table 1 lists a representative study that involves human participants for each of the above-mentioned paradigms. We evaluate SHARPIE’s functional expressiveness by using it to implement a replication for each of these studies. To assess the framework’s flexibility, we record the number of lines of code (LoC) (excluding docstrings, comments, and empty lines) and the number of experiment configuration database (DB) entries. In our analysis, we focus on the overhead imposed by creating a human-participant study and therefore only include code and configurations to correctly route policy decisions, participant inputs, agent outputs, and environment renderings to the appropriate location. That is, we explicitly do not incorporate agent training and environment simulation logic as this is out of scope for our framework. The remainder of this section consists of detailed descriptions of these replications per use case.

SayCan by [20] implements the first paradigm, where a human participant and RL agents interact by exchanging messages containing instructions. Specifically, the human user provides high-level, unstructured natural language instructions, which the agent then grounds into executable skills using learned affordances. The primary challenge in this paradigm is message passing: standard RL environments and agent APIs typically expect fixed-dimensional numerical vectors (e.g., standard Gym spaces). Routing variable-length string payloads from a user-facing web interface to the agent’s policy typically requires custom serialization and custom API wrappers. SHARPIE natively abstracts this exchange. We successfully connected a participant-facing text-input interface to a SayCan policy interacting with a table top pick-and-place environment using only 17 LoC and 5 DB entries. This demonstrates SHARPIE’s capability to support experiments involving messages between humans and RL agents.

Dagger [34] represents the second interaction paradigm, where a human expert provides corrective action labels for states visited by the agent’s current policy. To evaluate the framework’s support for this paradigm, we implemented an early Dagger-based imitation learning experiment using the *Super Mario Bros* environment [22]. A substantial proportion of the 43 LoC implementation deals with migrating the environment from an old version of OpenAI-Gym library and translating participant inputs to the environment’s action space. While the environment uses a bitmask-based input system to represent concurrent button presses (e.g., ‘right’ + ‘jump’), we found that mapping these inputs to the agent’s action space is straightforward using SHARPIE utilities. At the same time, SHARPIE’s policy interface enables the interception of the control loop to implement Dagger’s “query” phase for the expert policy. Here, our implementation routes the current observation to the participant, while the update method ensures that the resulting transition is correctly captured and buffered for the training phase. Overall, complete integration is done with 65 LoC and 5 DB entries.

Overcooked exemplifies paradigm three, where humans and RL agents act simultaneously within a shared environment. In the original study, a human participant plays cooperatively with an agent trained via either Self-Play or Behavioral Cloning (SP/BC) in the Overcooked environment [9]. The primary infrastructural challenge in this paradigm is real-time synchronization: the system must reliably stream environment renderings to the participant-facing web interface, capture participant keystrokes, query the RL policy, merge these separate actions, and transition the environment. Using SHARPIE, we replicated this functionality using 29 LoC and 6 DB entries, which highlights SHARPIE’s capacity to support experiments in which users and agents operate in realtime in the same environment. Access to the original agents³ would allow a full replication of the user study performed in the original paper.

³https://github.com/HumanCompatibleAI/overcooked_ai

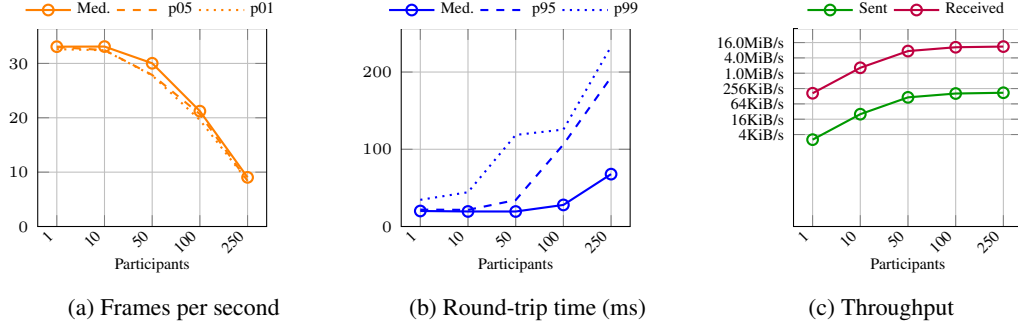


Figure 3: Scalability by number of participants.

TAMER represents the fourth paradigm, where human participants specify an immediate reward via evaluative feedback. In the TAMER framework [25], the participants observe the agent act and provide an asynchronous positive or negative label to its decisions. These labels are interpreted as immediate rewards and used to train a policy during interactions with the participant and the environment. Implementing this in a standard RL codebase often requires building complex threading or networking logic to inject asynchronous UI inputs into the agent’s synchronous training loop. SHARPIE’s functionalities reduce this overhead significantly. We successfully captured and routed the human’s real-time evaluative feedback for the MountainCar environment to the agent’s reward signal using only 31 LoC and 5 DB entries. This demonstrates that SHARPIE can elegantly handle the timing and synchronization challenges inherent to interactive reward shaping at low implementation cost.

5 Scalability and Performance Profiling

We now evaluate SHARPIE’s scalability and performance characteristics to ensure it meets the rigorous demands of interaction studies involving multiple agents and humans. In our benchmarks we include four aspects that might affect reliability of experiment results. First, we assess the scalability with respect to the number of human participants interacting with a single environment simultaneously to assess the degree to which SHARPIE can support experiments involving multiple simultaneous participants. Second, we assess the overhead imposed by SHARPIE and its effects on scalability with respect to the number of RL agents. Third, we assess performance in geographically distributed experimentation scenarios. Although SHARPIE’s architecture supports geographically distributed experiments due to its client-server setup with the environment simulation performed at a single server, this architecture may negatively affect performance in scenarios where participants’ connection is of low quality. Finally, we test how the framework is affected by environments with reasonably high computational demands, including environments producing high-resolution renderings.

In Figure 3 we present scalability and performance profiles for these scenarios for a target of 30 frames per second (FPS) for 20 trajectories of 100 time steps each and each with its own random seed, resulting in a total of 2 000 samples per experiment using several metrics. First, we consider median, 1st and 5th percentile (p01, p05) FPS as an indicator of performance for real-time interactions. Second, we report the median and 95th and 99th percentile (p95, p99) round-trip times (RTT) as the latency between a participant sending an action and receiving its associated observation. Third, we report throughput in bits as a measure of capacity of the system independent of the individual participant experience. We also recorded the frequency of socket-level errors or dropped packets which was 0 in all trials. All tests have been conducted using the same machine with one Intel® Core™ Ultra 9 (22 cores) and 96 GB of RAM.

5.1 Human participant scalability

This experiment assesses SHARPIE’s capacity to host large-scale participant studies. We simulate $n \in \{1, 10, 50, 100, 250\}$ simultaneous participants interacting simultaneously with a single real-time environment instance, in a single experiment, hosted at the same machine. To isolate SHARPIE overhead from environment-specific computation, we use a no-op environment which discards the N

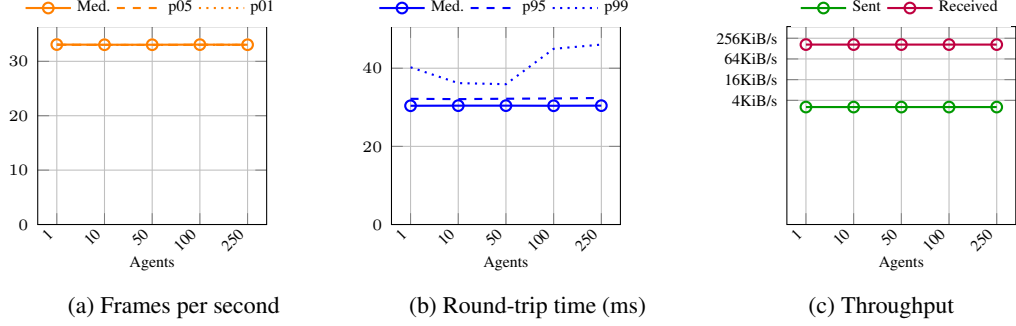


Figure 4: Scalability in the number of RL agents.

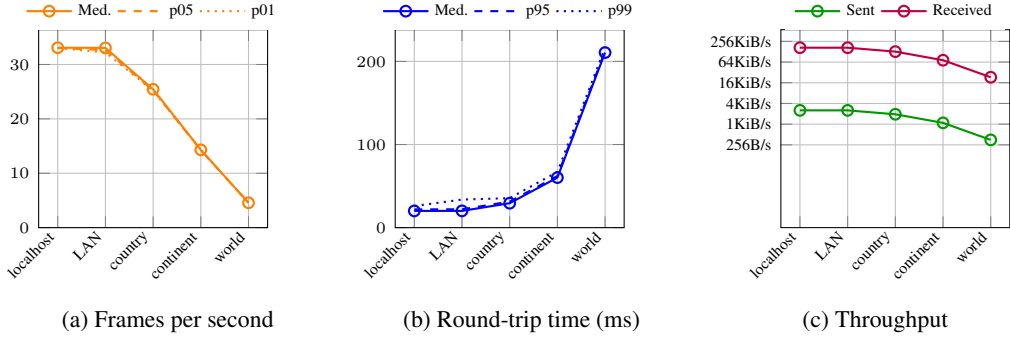


Figure 5: Performance stability analysis under variable network conditions.

received actions, and returns a 64×64 RGB observation containing random noise at each time step to simulate a visual payload per timestep. Having all participants interact with the same environment instance is challenging since all inputs need to be processed simultaneously before transitioning the environment to the next state. Moreover, this single-experiment setting is the most relevant since scalability in settings with multiple environment instances can be easily achieved by hosting separate experiments on separate SHARPIE web server instances, i.e. by scaling horizontally with n .

From Figure 3 we see that the system maintains nominal performance up to $n = 50$ concurrent participants in real-time interactions, with median FPS remaining close to the target of 30 and median RTT showing negligible variance across number of participants ($\Delta \approx 0.1\text{ms}$). Although p95 and p99 RTT increase substantially for $n = 100$ participants, median FPS remains reasonable. At $n = 250$, however, the system has reached a performance bottleneck and median FPS drops below 10, while median RTT increases to 67.80 ms. This non-linear degradation indicates a saturation point of the single-instance server architecture under the tested hardware configuration. Notably, across all tested scales, the system recorded zero communication errors, indicating that while throughput limits affect fluidity and will affect experimental results, the underlying state synchronization remains intact.

5.2 Multi-agent scalability

Similarly, we evaluate SHARPIE’s overhead in scaling the number of RL agents. We simulate $n \in \{1, 10, 50, 100, 250\}$ concurrent random RL agents interacting in a single no-op environment instance. This environment discards all actions and samples a random 64×64 noise image at each timestep. The metrics reported in Figure 4 show that the system maintains nominal performance in all cases. We hypothesize that this is due to SHARPIE’s architecture, in which all RL policies run in the same process as the environment simulation loop to minimize data transfer and the overhead imposed by SHARPIE consists mostly of routing and logging of messages.

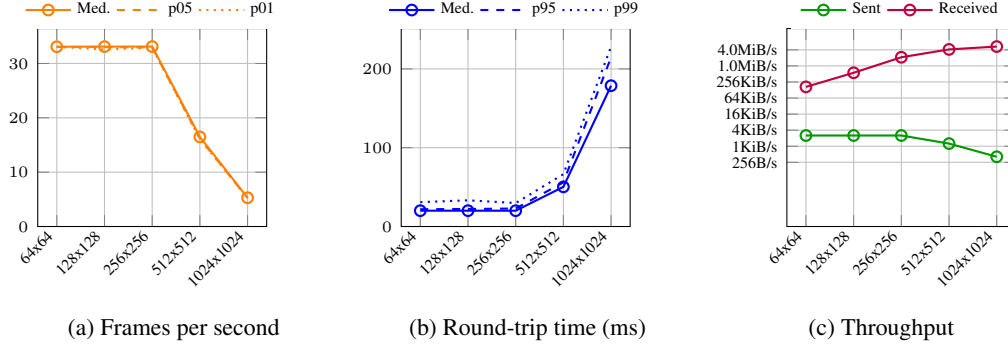


Figure 6: Performance metrics for a single participant under varying image size.

5.3 Network Quality and Geographic Distribution

To assess the suitability of our server-client architecture in supporting experiments involving geographically distant participants and RL agents, we simulate the impact of network-induced latency on system performance. Specifically, we emulate various network latencies to assess FPS and RTT in the following settings: a SHARPIE webserver instance running on the participants' localhost (<0.1 ms), on the participants network (<10 ms), in the participants country (<20 ms), on the participants continent (50 ms) and world-wide (200 ms). The results reported in Figure 5 show that, with a target of 30 FPS, SHARPIE does not suffer loss when running on the same LAN. This is perfect for hosting interactive real-time experiments in the same building as the server running SHARPIE. For an experiment requiring participants from a cloud source platform, the system will be able to run at least 26 FPS and provide a smooth interaction. However, for regional or world-wide interactions, it is more suited for turn-based rather than (near-) real-time experiments.

5.4 Environment and Rendering Complexity

Finally, we test the throughput of the framework when handling high definition rendering or highly demanding payloads. To illustrate that, we evaluate rendering different image sizes with random noise as well as testing a Starcraft II environment. The results in Figure 6 show that SHARPIE can handle up to 256x256 payloads at 30FPS which will comfortably serve most use cases. Scalability beyond this size remains challenging and will require additional measures such as compression currently not implemented in SHARPIE.

6 Discussion & Future Work

We have presented and motivated the design and implementation of a framework to accelerate human-AI interaction studies, with a particular focus on studies for multi-agent tasks involving humans. As stated, the scope of this project is not to implement alternative environment and algorithms for such studies, but rather to integrate with existing solutions seamlessly and provide utilities to ease experimentation with participants. This is done by empowering researchers through a modular software interface relying, in part, on the de facto standard set by Gymnasium. With SHARPIE we aim to provide an easily integrable framework that researchers can use to painlessly set up experiments involving both human and artificial agents. Our hope is that in turn such an architecture lays the foundation for a standard for the interaction between human and artificial agents.

This modular approach also allows for numerous directions of improvement in terms of interoperability and features. In the former case, we aim to provide an increasing number of ready-made, supported plugins to handle a large part of the existing work on environments, libraries, and deployment options. In the latter case, we plan to widen the scope of possible human-agent interactions by incorporating additional modalities such as audio or video [11, 24]. The resulting real-time and multimodal communication between users and agents would allow the study of rich and fine-grained communication protocols, and support experiments involving pitch and tone, nonverbal cues, etc. Finally, a hosted version of SHARPIE has been used for outreach, education and user literacy purposes.

Acknowledgements

We thank Andreas Sauter and Mustafa Mert Çelikok for their input on an early design of SHARPIE. This research was funded by the Hybrid Intelligence Centre, a 10-year programme supported by the Dutch Ministry of Education, Culture and Science through the Netherlands Organisation for Scientific Research (NWO), under grant number 024.004.022.

References

- [1] Zeynep Akata, Dan Balliet, Maarten De Rijke, Frank Dignum, Virginia Dignum, Gusztai Eiben, Antske Fokkens, Davide Grossi, Koen Hindriks, Holger Hoos, et al. A research agenda for hybrid intelligence: augmenting human intellect with collaborative, adaptive, responsible, and explainable artificial intelligence. *Computer*, 53(8):18–28, 2020.
- [2] Nele Albers, Mark A Neerincx, and Willem-Paul Brinkman. Addressing people’s current and future states in a reinforcement learning algorithm for persuading to quit smoking and to be physically active. *Plos one*, 17(12), 2022.
- [3] Lucas N Alegre, Florian Felten, El-Ghazali Talbi, Grégoire Danoy, Ann Nowé, Ana LC Bazzan, and Bruno C da Silva. Mo-gym: A library of multi-objective reinforcement learning environments. In *Proceedings of the 34th Benelux Conference on Artificial Intelligence BNAIC/Benelearn*, 2022.
- [4] Kim Baraka, Ifrah Idrees, Taylor Kessler Faulkner, Erdem Biyik, Serena Booth, Mohamed Chetouani, Daniel H. Grollman, Akanksha Saran, Emmanuel Senft, Silvia Tulli, Anna-Lisa Vollmer, Antonio Andriella, Helen Beierling, Tiffany Horter, Jens Kober, Isaac Sheidlower, Matthew E. Taylor, Sanne van Waveren, and Xuesu Xiao. Human-interactive robot learning: Definition, challenges, and recommendations. *Journal of Human-Robot Interaction*, 15(2), February 2026. doi: 10.1145/3779297.
- [5] Kim Baraka, Ifrah Idrees, Taylor Kessler Faulkner, Erdem Biyik, Serena Booth, Mohamed Chetouani, Daniel H Grollman, Akanksha Saran, Emmanuel Senft, Silvia Tulli, et al. Human-interactive robot learning: Definition, challenges, and recommendations. *ACM Transactions on Human-Robot Interaction*, 15(2):1–31, 2026.
- [6] Erdem Biyik, Aditi Talati, and Dorsa Sadigh. Aprel: A library for active preference-based reward learning algorithms. In *2022 17th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, pages 613–617. IEEE, 2022.
- [7] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal Maclaurin, George Nectra, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/jax-ml/jax>.
- [8] G Brockman. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.
- [9] Micah Carroll, Rohin Shah, Mark K. Ho, Thomas L. Griffiths, Sanjit A. Seshia, Pieter Abbeel, and Anca Dragan. On the utility of learning about humans for human-ai coordination. In *Advances in Neural Information Processing Systems*, 2019.
- [10] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in Neural Information Processing Systems*, 2017.
- [11] Konstantinos Christofi and Kim Baraka. Uncovering patterns in humans that teach robots through demonstrations and feedback. In *Companion of the 2024 ACM/IEEE International Conference on Human-Robot Interaction*, pages 332–336, 2024.
- [12] Joshua R De Leeuw. jspsych: A javascript library for creating behavioral experiments in a web browser. *Behavior research methods*, 47:1–12, 2015.

- [13] Florian Felten, Lucas N. Alegre, Ann Nowé, Ana L. C. Bazzan, El Ghazali Talbi, Grégoire Danoy, and Bruno Castro da Silva. A toolkit for reliable benchmarking and research in multi-objective reinforcement learning. In *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS 2023)*, 2023.
- [14] Florian Felten, Umut Ucak, Hicham Azmani, Gao Peng, Willem Röpke, Hendrik Baier, Patrick Mannion, Diederik M Roijers, Jordan K Terry, El-Ghazali Talbi, et al. Momaland: A set of benchmarks for multi-objective multi-agent reinforcement learning. In *Multi-objective Decision Making Workshop at ECAI 2024*, 2024.
- [15] Jason Gauci, Edoardo Conti, Yitao Liang, Kittipat Virochsiri, Yuchen He, Zachary Kaden, Vivek Narayanan, Xiaohui Ye, Zhengxing Chen, and Scott Fujimoto. Horizon: Facebook’s open source applied reinforcement learning platform. In *ICML 2019 Workshop on RL4RealLife*, 2019.
- [16] Kevin Godin-Dubois, Karine Miras, and Anna V Kononova. AMaze: A benchmark generator for sighted maze-navigating agents. *Journal of Open Source Software*, 2024.
- [17] Paul Green and Lisa Wei-Haas. The rapid development of user interfaces: Experience with the wizard of oz method. *Proceedings of the Human Factors Society Annual Meeting*, 29(5): 470–474, 1985. doi: 10.1177/154193128502900515. URL <https://doi.org/10.1177/154193128502900515>.
- [18] Lin Guan, Mudit Verma, Suna Sihang Guo, Ruohan Zhang, and Subbarao Kambhampati. Widening the pipeline in human-guided reinforcement learning with explanation and context-aware data augmentation. *Advances in Neural Information Processing Systems*, 2021.
- [19] Shengyi Huang, Rousslan Fernand Julien Dossa, Chang Ye, Jeff Braga, Dipam Chakraborty, Kinal Mehta, and João G.M. Araújo. CleanRL: High-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research*, 23(274):1–18, 2022. URL <http://jmlr.org/papers/v23/21-1342.html>.
- [20] Brian Ichter, Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, Ryan Julian, Dmitry Kalashnikov, Sergey Levine, Yao Lu, Carolina Parada, Kanishka Rao, Pierre Sermanet, Alexander T Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Mengyuan Yan, Noah Brown, Michael Ahn, Omar Cortes, Nicolas Sievers, Clayton Tan, Sichun Xu, Diego Reyes, Jarek Rettinghouse, Jornell Quiambao, Peter Pastor, Linda Luu, Kuang-Huei Lee, Yuheng Kuang, Sally Jesmonth, Nikhil J. Joshi, Kyle Jeffrey, Rosario Jauregui Ruano, Jasmine Hsu, Keerthana Gopalakrishnan, Byron David, Andy Zeng, and Chuyuan Kelly Fu. Do as i can, not as i say: Grounding language in robotic affordances. In Karen Liu, Dana Kulic, and Jeff Ichnowski, editors, *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318, 14–18 Dec 2023.
- [21] Timo Kaufmann, Paul Weng, Viktor Bengs, and Eyke Hüllermeier. A survey of reinforcement learning from human feedback. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=f70kIurx4b>.
- [22] Christian Kauten. Super Mario Bros for OpenAI Gym. GitHub, 2018. URL <https://github.com/Kautenja/gym-super-mario-bros>.
- [23] Matilda Knierim, Sahil Jain, Murat Han Aydoğan, Kenneth Mitra, Kush Desai, Akanksha Saran, and Kim Baraka. Prosody as a teaching signal for agent learning: Exploratory studies and algorithmic implications. *arXiv preprint arXiv:2410.23554*, 2024.
- [24] Matilda Knierim, Sahil Jain, Murat Han Aydoğan, Kenneth D Mitra, Kush Desai, Akanksha Saran, and Kim Baraka. Leveraging prosody as an informative teaching signal for agent learning: Exploratory studies and algorithmic implications. In *Proceedings of the 26th International Conference on Multimodal Interaction*, pages 95–123, 2024.
- [25] W Bradley Knox and Peter Stone. Interactively shaping agents via human reinforcement: The tamer framework. In *Proceedings of the fifth international conference on Knowledge capture*, pages 9–16, 2009.

- [26] Marc Lanctot, Edward Lockhart, Jean-Baptiste Lespiau, Vinicius Zambaldi, Satyaki Upadhyay, Julien Pérolat, Sriram Srinivasan, Finbarr Timbers, Karl Tuyls, Shayegan Omidshafiei, et al. OpenSpiel: A framework for reinforcement learning in games. *arXiv preprint arXiv:1908.09453*, 2019.
- [27] Eric Liang, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph Gonzalez, Michael Jordan, and Ion Stoica. RLlib: Abstractions for distributed reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning*, 2018.
- [28] Chase McDonald and Cleotilde Gonzalez. Cogrid & the multi-user gymnasium: A framework for multi-agent experimentation. *arXiv preprint arXiv:2604.15044*, 2026.
- [29] Sriraam Natarajan, Gautam Kunapuli, Kshitij Judah, Prasad Tadeipalli, Kristian Kersting, and Jude Shavlik. Multi-agent inverse reinforcement learning. In *2010 ninth international conference on machine learning and applications*, pages 395–400. IEEE, 2010.
- [30] Jonathan W Peirce. Psychopy—psychophysics software in python. *Journal of neuroscience methods*, 162(1-2):8–13, 2007.
- [31] Cheng Qian, Zuxin Liu, Akshara Prabhakar, Jieli Qiu, Zhiwei Liu, Haolin Chen, Shirley Kokane, Heng Ji, Weiran Yao, Shelby Heinecke, et al. Userrl: Training interactive user-centric agent via reinforcement learning. *arXiv preprint arXiv:2509.19736*, 2025.
- [32] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.
- [33] Carl Orge Retzlaff, Srijita Das, Christabel Wayllace, Payam Mousavi, Mohammad Afshari, Tianpei Yang, Anna Saranti, Alessa Angerschmid, Matthew E Taylor, and Andreas Holzinger. Human-in-the-loop reinforcement learning: A survey and position on requirements, challenges, and opportunities. *Journal of Artificial Intelligence Research*, 79:359–415, 2024.
- [34] Stephane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, Proceedings of Machine Learning Research, pages 627–635, 2011.
- [35] Alexander Rutherford, Benjamin Ellis, Matteo Gallici, Jonathan Cook, Andrei Lupu, Garðar Ingvarsson, Timon Willi, Akbir Khan, Christian Schroeder de Witt, Alexandra Souly, Saptarashmi Bandyopadhyay, Mikayel Samvelyan, Minqi Jiang, Robert Lange, Shimon Whiteson, Bruno Lacerda, Nick Hawes, Tim Rocktäschel, Chris Lu, and Jakob Foerster. Jaxmarl: Multi-agent rl environments and algorithms in jax. In *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*, Richland, SC, 2024. International Foundation for Autonomous Agents and Multiagent Systems. ISBN 9798400704864.
- [36] William Saunders, Girish Sastry, Andreas Stuhlmüller, and Owain Evans. Trial without error: Towards safe reinforcement learning via human intervention. In *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*, 2018.
- [37] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 1998.
- [38] Matthew E Taylor, Nicholas Nissen, Yuan Wang, and Neda Navidi. Improving reinforcement learning with human assistance: an argument for human subject studies with hippo gym. *Neural Computing and Applications*, 35(32), 2023.
- [39] Jordan Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, et al. Pettingzoo: Gym for multi-agent reinforcement learning. *Advances in Neural Information Processing Systems*, 34:15032–15043, 2021.

- [40] Marcel Torne, Max Balsells, Zihan Wang, Samedh Desai, Tao Chen, Pulkit Agrawal, and Abhishek Gupta. Breadcrumbs to the goal: goal-conditioned exploration from human-in-the-loop feedback. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, pages 63222–63258, 2023.
- [41] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulao, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.
- [42] Yifu Yuan, Jianye Hao, Yi Ma, Zibin Dong, Hebin Liang, Jinyi Liu, Zhixin Feng, Kai Zhao, and Yan Zheng. Uni-RLHF: Universal platform and benchmark suite for reinforcement learning with diverse human feedback. In *The Twelfth International Conference on Learning Representations, ICLR*, 2024.
- [43] Yangyang Zhao, Zhenyu Wang, Changxi Zhu, and Shihan Wang. Efficient dialogue complementary policy learning via deep q-network policy and episodic memory policy. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 4311–4323, 2021.
- [44] Lianmin Zheng, Jiacheng Yang, Han Cai, Ming Zhou, Weinan Zhang, Jun Wang, and Yong Yu. Magent: A many-agent reinforcement learning platform for artificial collective intelligence. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- [45] Zheqing Zhu, Rodrigo de Salvo Braz, Jalaj Bhandari, Daniel Jiang, Yi Wan, Yonathan Efroni, Liyuan Wang, Ruiyang Xu, Hongbo Guo, Alex Nikulkov, et al. Pearl: A production-ready reinforcement learning agent. *Journal of Machine Learning Research*, 25(273):1–30, 2024.

A APIs in major RL packages

Name	Description	Language	Target	API
Amaze ¹	Maze navigation environments	Python	Research	Gymnasium+
CleanRL ²	Algorithm library	Python/Torch	Research/Production	Gymnasium
Gymnasium ³	Standard environments	Python	Research	Gymnasium
HIPPO-Gym ⁴	Human Input Parsing	Python	Research	Gymnasium+
Interactive-Gym	Interactive experiment platform	Python	Research	Gymnasium+
JaxMARL ⁵	MA environments	JAX	Research	Gymnasium+
Momaland ⁶	MA MO environments	Python	Research	Gymnasium
MO-Gymnasium ⁷	MO environments	Python	Research	Gymnasium+
MORL Baselines ⁸	MO algorithm library	Python	Research	Gymnasium+
OpenSpiel ⁹	Game (theory) envs/algorithms	Python	Research	Gymnasium
Pearl ¹⁰	Algorithm library	Python	Production	Gymnasium
PettingZoo ¹¹	MA environments	Python	Research	Gymnasium+
Ray RLLib ¹²	Algorithm library	Python/Torch	Production	Gymnasium
RLax ¹³	Algorithm library	JAX	Research	Gymnasium
Stable-baselines3 ¹⁴	Algorithm library	Python/Torch	Research/Production	Gymnasium
Uni-RLHF ¹⁵	Annotation tool	Python	Annotation	Gymnasium+

Table 2: Comparison of RL Packages, APIs, and Applications. + indicates minor alterations to the Gymnasium API such as a vectorial reward for multi-objective, or vectorial action for multi-agent RL. References are: 1- Godin-Dubois et al. [16]; 2- Huang et al. [19]; 3- Brockman [8]; 4- Taylor et al. [38]; 5- Rutherford et al. [35]; 6-Felten et al. [14]; 7- Alegre et al. [3]; 8- Felten et al. [13]; 9- Lanctot et al. [26]; 10- Zhu et al. [45]; 11- Terry et al. [39]; 12- Liang et al. [27]; 13- Bradbury et al. [7]; 14- Raffin et al. [32]; 15-Yuan et al. [42]. MA: multi-agent, MO; multi-objective, env: environment.

B SHARPIE data model

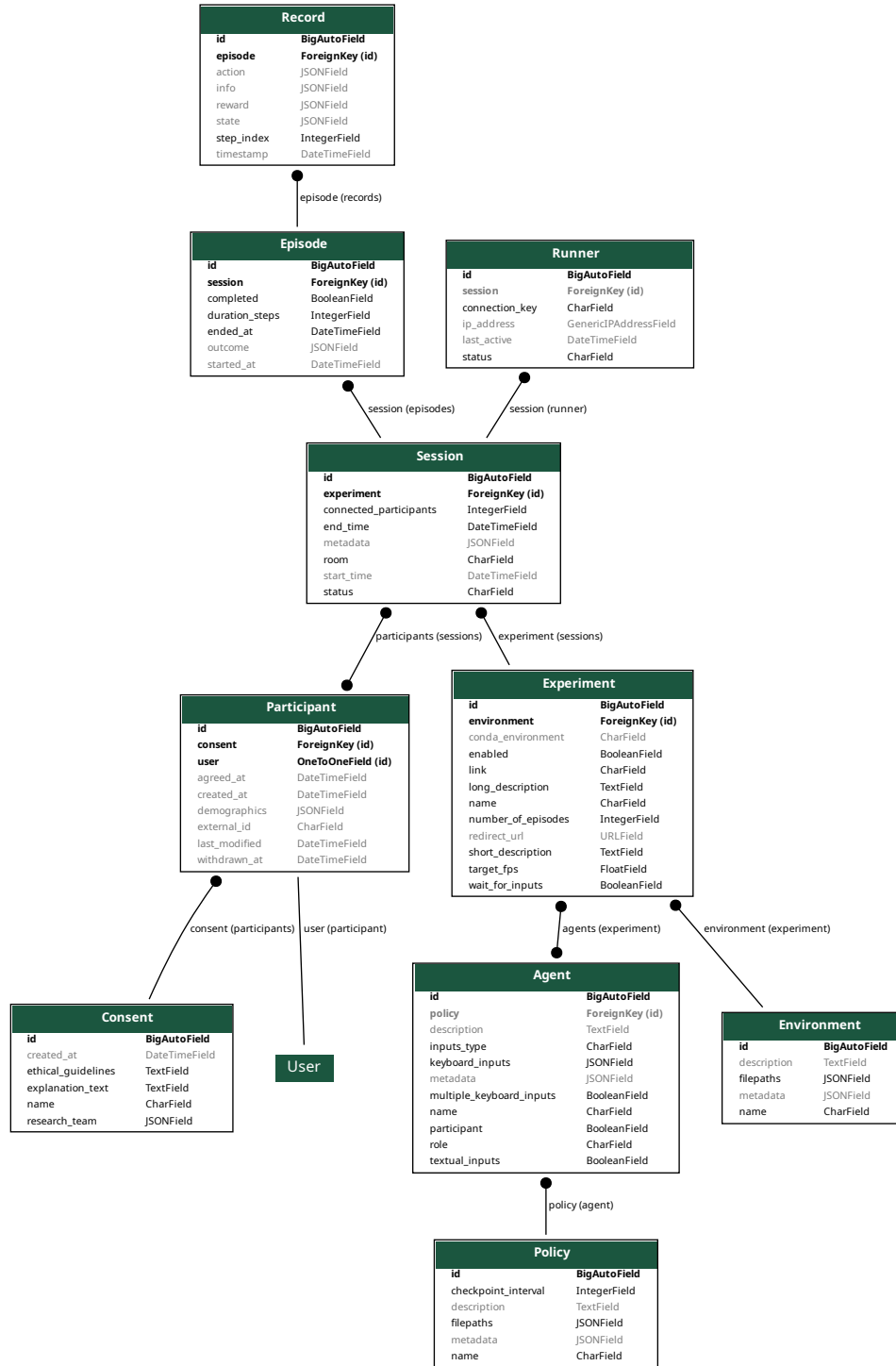


Figure 7: Overview of SHARPIE's data model

C Tabular scalability benchmark results

Participants	FPS	RTT	p95 RTT	Sent	Received
1	33.08 (100.0%)	20.08 (100.0%)	21.87 (100.0%)	3KiB/s (100.0%)	168KiB/s (100.0%)
10	33.10 (100.1%)	19.48 (97.0%)	21.55 (98.5%)	25KiB/s (1000.4%)	1.6MiB/s (1000.4%)
50	30.03 (90.8%)	19.42 (96.7%)	34.24 (156.5%)	115KiB/s (4582.5%)	7.4MiB/s (4535.2%)
100	21.22 (64.2%)	27.93 (139.1%)	106.20 (485.6%)	163KiB/s (6483.3%)	10.5MiB/s (6409.4%)
250	9.04 (27.3%)	67.80 (337.7%)	193.22 (883.5%)	175KiB/s (6952.7%)	11.1MiB/s (6813.4%)

Table 3: Performance metrics by participant count, results in parenthesis are relative to $N = 1$ baseline.

Agents	FPS	RTT	p95 RTT	Sent	Received
1	33.08 (100.0%)	30.39 (100.0%)	32.17 (100.0%)	3KiB/s (100.0%)	168KiB/s (100.0%)
10	33.05 (99.9%)	30.42 (100.1%)	32.09 (99.7%)	3KiB/s (99.9%)	167KiB/s (99.9%)
50	33.05 (99.9%)	30.42 (100.1%)	32.19 (100.1%)	3KiB/s (99.9%)	167KiB/s (99.9%)
100	33.07 (100.0%)	30.39 (100.0%)	32.26 (100.3%)	3KiB/s (100.0%)	167KiB/s (100.0%)
250	33.05 (99.9%)	30.42 (100.1%)	32.38 (100.6%)	3KiB/s (99.9%)	167KiB/s (99.9%)

Table 4: Performance metrics for a variable number of RL agents, results in parenthesis are relative to $N = 1$ baseline.

Latency ms	FPS	RTT	p95 RTT	Sent	Received
0.1	33.09 (100.0%)	20.09 (100.0%)	21.94 (100.0%)	3KiB/s (100.0%)	168KiB/s (100.0%)
10.0	33.05 (99.9%)	20.14 (100.3%)	22.08 (100.6%)	3KiB/s (99.9%)	167KiB/s (99.9%)
20.0	25.44 (76.9%)	29.38 (146.3%)	31.25 (142.4%)	2KiB/s (76.9%)	129KiB/s (76.9%)
50.0	14.31 (43.2%)	60.21 (299.8%)	62.05 (282.8%)	1KiB/s (43.2%)	72KiB/s (43.2%)
200.0	4.57 (13.8%)	210.63 (1048.7%)	212.50 (968.4%)	357B/s (13.8%)	23KiB/s (13.8%)

Table 5: Performance stability analysis under variable network conditions, results in parenthesis are relative to localhost baseline.

Render size	FPS	RTT	p95 RTT	Sent	Received
64x64	33.08 (100.0%)	20.07 (100.0%)	21.96 (100.0%)	3KiB/s (100.0%)	168KiB/s (100.0%)
128x128	33.10 (100.1%)	20.09 (100.1%)	22.20 (101.1%)	3KiB/s (100.1%)	567KiB/s (338.5%)
256x256	33.11 (100.1%)	20.03 (99.8%)	22.84 (104.0%)	3KiB/s (100.1%)	2.1MiB/s (1292.7%)
512x512	16.49 (49.8%)	50.31 (250.7%)	56.02 (255.1%)	1KiB/s (49.8%)	4.2MiB/s (2543.9%)
1024x1024	5.29 (16.0%)	178.70 (890.4%)	214.89 (978.4%)	412B/s (16.0%)	5.3MiB/s (3252.9%)

Table 6: Performance metrics for a single participant under varying image size, results in parenthesis are relative to the smallest (64x64).